

Experiment-05: Missionaries and Cannibals

Problem using Python

Aim

To develop a Python program that solves the **Missionaries and Cannibals Problem** using the **Breadth-First Search (BFS)** algorithm and finds a safe sequence of moves to transport all missionaries and cannibals across the river.

Problem Statement

There are:

- **3 Missionaries (M)**
- **3 Cannibals (C)**
- **1 Boat** (capacity = 2)

Rules

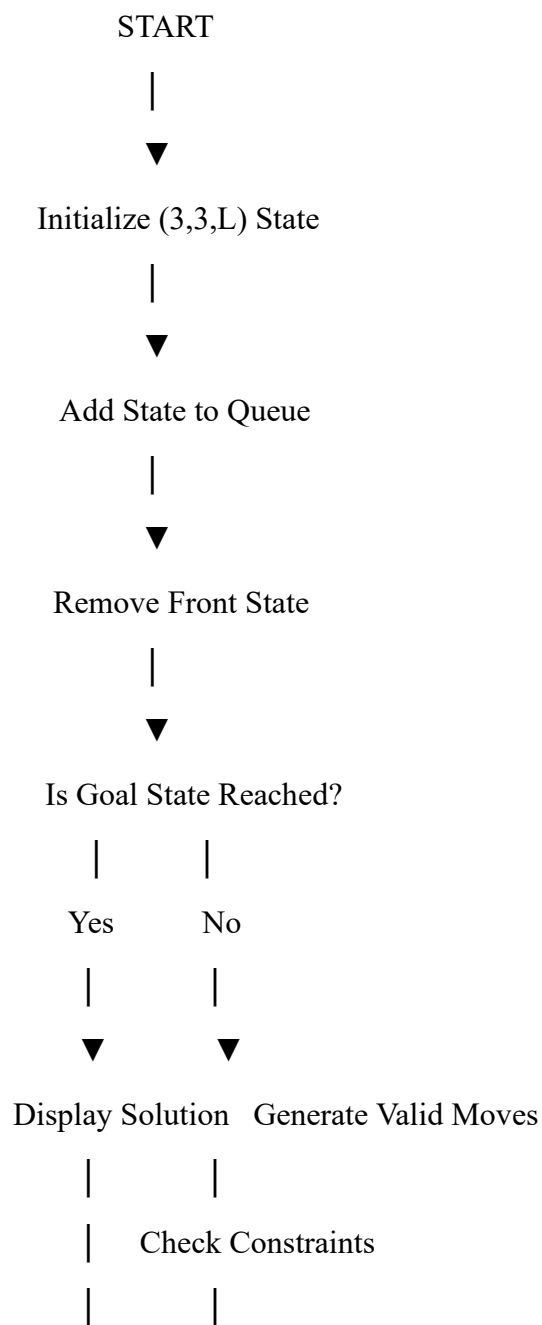
- The boat can carry **at most 2 people**.
 - The boat cannot cross the river empty.
 - At no bank should the number of cannibals exceed the number of missionaries (unless there are no missionaries on that bank).
 - Move all missionaries and cannibals safely from the left bank to the right bank.
-

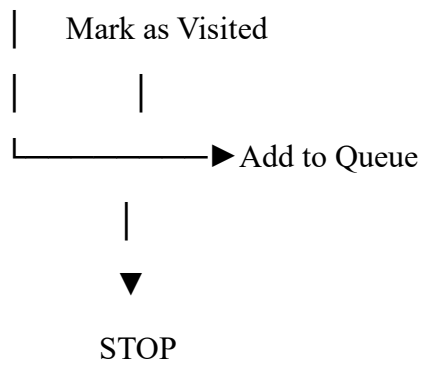
Algorithm

1. Represent each state as (**Missionaries Left, Cannibals Left, Boat Position**).
2. Start from the initial state (**3,3,'L'**).
3. Use a queue to perform **Breadth-First Search (BFS)**.
4. Generate all possible boat moves:
 - (2 Missionaries)
 - (2 Cannibals)
 - (1 Missionary + 1 Cannibal)
 - (1 Missionary)
 - (1 Cannibal)

5. Check whether the new state is valid:
 - Missionaries are never outnumbered by cannibals on either bank.
 - Values remain between 0 and 3.
 6. Ignore already visited states.
 7. Continue until the goal state **(0,0,'R')** is reached.
 8. Display the sequence of safe moves.
-

Flowchart





Python Code

```
from collections import deque
```

```
# Possible boat moves
```

```
moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]
```

```
def is_valid(m_left, c_left):
```

```
    m_right = 3 - m_left
```

```
    c_right = 3 - c_left
```

```
    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
```

```
        return False
```

```
    if m_left > 0 and c_left > m_left:
```

```
        return False
```

```
    if m_right > 0 and c_right > m_right:
```

```
        return False
```

```
    return True
```

```
def bfs():
```

```
start = (3, 3, 'L')
```

```
goal = (0, 0, 'R')
```

```
queue = deque([(start, [])])
```

```
visited = set()
```

```
while queue:
```

```
    (m, c, boat), path = queue.popleft()
```

```
    if (m, c, boat) in visited:
```

```
        continue
```

```
    visited.add((m, c, boat))
```

```
    path = path + [(m, c, boat)]
```

```
    if (m, c, boat) == goal:
```

```
        return path
```

```
    for dm, dc in moves:
```

```
        if boat == 'L':
```

```
            new_state = (m - dm, c - dc, 'R')
```

```
        else:
```

```
            new_state = (m + dm, c + dc, 'L')
```

```
        if is_valid(new_state[0], new_state[1]):
```

```
            queue.append((new_state, path))
```

```
return None
```

```
solution = bfs()
```

```
if solution:
```

```
    print("Solution Path:\n")
```

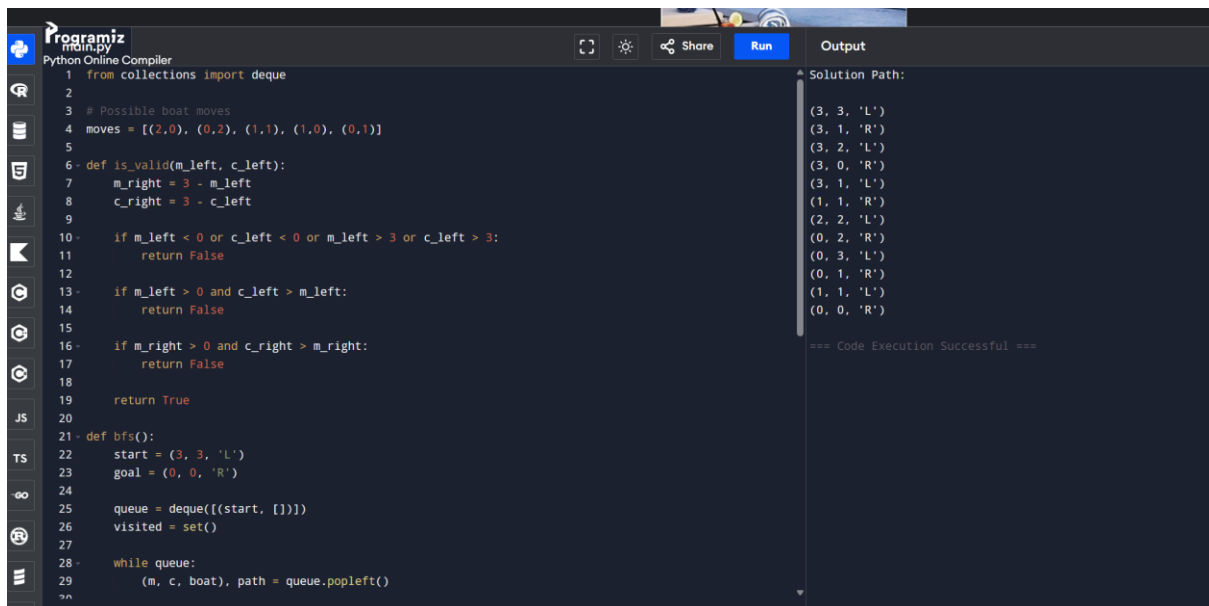
```
    for state in solution:
```

```
        print(state)
```

```
else:
```

```
    print("No solution found.")
```

Sample Output



The screenshot shows a Python Online Compiler interface with a dark theme. The code editor on the left contains a Python script for solving the Missionaries and Cannibals Problem using Breadth-First Search (BFS). The script defines a set of possible moves, a validation function to ensure missionaries are never outnumbered by cannibals, and a BFS function that finds the solution path. The output panel on the right displays the solution path as a sequence of states (missionaries, cannibals, boat position) and the moves taken to reach the goal state (0,0,'R'). The output is as follows:

```
Solution Path:
(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')

=== Code Execution Successful ===
```

Result

The **Missionaries and Cannibals Problem** was successfully solved using the **Breadth-First Search (BFS)** algorithm. The program generated a valid sequence of moves while ensuring that missionaries were never outnumbered by cannibals on either bank, and safely reached the goal state **(0,0,'R')**.